

EE673 Assignment 2

Mohd Amir, 220660

17th February, 2026

1 TCP and UDP Labs

1.1 TCP Lab

1. Source IP: 192.168.1.102:1161 (PORT - 1161)
2. Destination IP: 128.119.245.12:80 (PORT - 80)
3. Client Computer (Source) IP: 172.23.5.133:51222 (PORT - 51222)
4. (a) **Sequence Number of the TCP SYN Segment: 0** (Relative).
(b) **Identification of the SYN Segment:** The segment is identified as a SYN segment because the **SYN bit** in the Flags field of the TCP header is set to 1. Specifically, the Flags field value is 0x002 (SYN).
5. (a) **Sequence Number of the SYNACK segment: 0** (Relative).
(b) **Acknowledgment Field Value: 1.**
(c) **Determination of the Acknowledgement Value:** gaia.cs.umass.edu determined this value by taking the sequence number of the initial **SYN** segment received from the client and adding 1 to it ($ISN_{client} + 1$). This serves as an acknowledgment that the SYN was received and indicates the next sequence number the server expects to receive from the client.
(d) **Identification of the SYNACK Segment:** The segment is identified as a SYNACK segment because both the **SYN** bit and the **ACK** bit in the Flags field of the TCP header are set to 1. The Flags field value is typically 0x012.
6. **Sequence Number of the TCP segment containing the HTTP POST: 1** (Relative) or **232129013** (Raw).

7. TCP Sequence Numbers and RTT Analysis

- **Sequence Numbers:** 1, 566, 2026, 3486, 4946, 6406

Table 1: RTT and Estimated RTT for the first six TCP segments

Segment	Sent Time (s)	ACK Received Time (s)	Sample RTT (s)	Estimated RTT (s)
1	0.026477	0.053937	0.027460	0.027460
2	0.041737	0.077294	0.035557	0.028472
3	0.054026	0.124085	0.070059	0.033670
4	0.054690	0.169118	0.114428	0.043765
5	0.077405	0.217299	0.139894	0.055781
6	0.078157	0.267802	0.189645	0.072514

Calculations: The *SampleRTT* is the time elapsed between the transmission of a segment and the receipt of its acknowledgement. The *EstimatedRTT* is updated using an Exponential Moving Average (EWMA) with a smoothing factor $\alpha = 0.125$:

$$\text{EstimatedRTT}_n = (1 - \alpha) \cdot \text{EstimatedRTT}_{n-1} + \alpha \cdot \text{SampleRTT}_n \quad (1)$$

For the initial segment ($n = 1$), the *EstimatedRTT* is set equal to the *SampleRTT* (0.027460 s).

8. **Length of the First Six TCP Segments:** The length of each segment is determined by the amount of application-layer data contained within the TCP payload. This can be found in the *TCP Segment Len* field or by calculating the difference between the sequence number of a segment and the sequence number of the subsequent segment.

- **Segment 1 (POST):** 565 bytes (566 - 1)
- **Segment 2:** 1460 bytes (2026 - 566)
- **Segment 3:** 1460 bytes (3486 - 2026)
- **Segment 4:** 1460 bytes (4946 - 3486)
- **Segment 5:** 1460 bytes (6406 - 4946)
- **Segment 6:** 1460 bytes (7866 - 6406)

9. Receiver Buffer Space and Sender Throttling

- **Minimum Advertised Buffer Space:** The minimum amount of available buffer space advertised by `gaia.cs.umass.edu` for the entire trace was **5840** bytes. This was found by inspecting the *Window* field in the TCP segments sent from the server to the client.
- **Sender Throttling:** The lack of receiver buffer space **did not** throttle the sender. *Reasoning:* The advertised window size never dropped to zero, and it remained consistently larger than the maximum segment size (MSS) of 1460 bytes. Therefore, the sender was able to transmit data without being restricted by the receiver's buffer capacity.

The minimum advertised receiver window is 5840 bytes and grows to 62780 bytes. The sender is never throttled.

10. Retransmitted Segments in the Trace

- **Findings:** There are **no** retransmitted segments in the trace file.
- **Verification Method:** To answer this question, I checked for the following:
 - (a) Applied the Wireshark display filter `tcp.analysis.retransmission` to isolate any packets identified by the analyzer as retransmissions.
 - (b) Inspected the *Expert Information* tool under the *Analyze* menu to look for any warnings regarding "Fast Retransmissions" or "Out-of-Order" packets.
 - (c) Verified that the sequence numbers for all segments sent from the client to the server were monotonically increasing, meaning no sequence number was repeated.

11. Receiver Acknowledgment Patterns

	ACKd Sequence Number	ACKd Data
ACK1	566	566
ACK2	2026	1460
ACK3	3486	1460
ACK4	4946	1460
ACK5	6046	1460
ACK6	7866	1460
ACK7	9013	1460
ACK8	10473	1460
ACK9	11933	1460
ACK10	13393	1460
ACK11	14853	1460
ACK12	16313	1460

Analysis: As shown in the table, the receiver typically acknowledges **1460 bytes** of data in each ACK, which corresponds to the Maximum Segment Size (MSS). In this specific trace, there are **no evident cases** of the receiver ACKing every other segment, as each ACK corresponds to exactly one full-sized data segment (1460 bytes). If the receiver were ACKing every other segment (Delayed ACK), the “ACKd Data” value would consistently be **2920 bytes**.

12. TCP Connection Throughput

- **Calculation Method:** Throughput is calculated as the total amount of data transferred divided by the total time taken for the transfer. The total data transferred was determined by taking the final Acknowledgment number from the server (164091) and subtracting the initial sequence number (1). The total time was calculated by taking the timestamp of the final ACK received from the server and subtracting the timestamp of the initial HTTP POST segment transmission.
- **Values Used:**
 - **Final ACK Received Time (Packet 202):** 5.455830 s
 - **Initial Segment Sent Time (Segment 1):** 0.026477 s
 - **Total Time:** 5.455830 – 0.026477 = 5.429353 s
 - **Total Data Transferred:** 164091 – 1 = 164090 bytes
- **Calculated Throughput:**

$$\text{Throughput} = \frac{164090 \text{ bytes}}{5.429353 \text{ seconds}} \approx 30222.75 \text{ bytes/sec} \quad (2)$$

Multiplying by 8 and dividing by 1000 gives a throughput of approximately **241.78 Kbps**.

13. TCP Congestion Control: Time-Sequence Graph (Stevens)

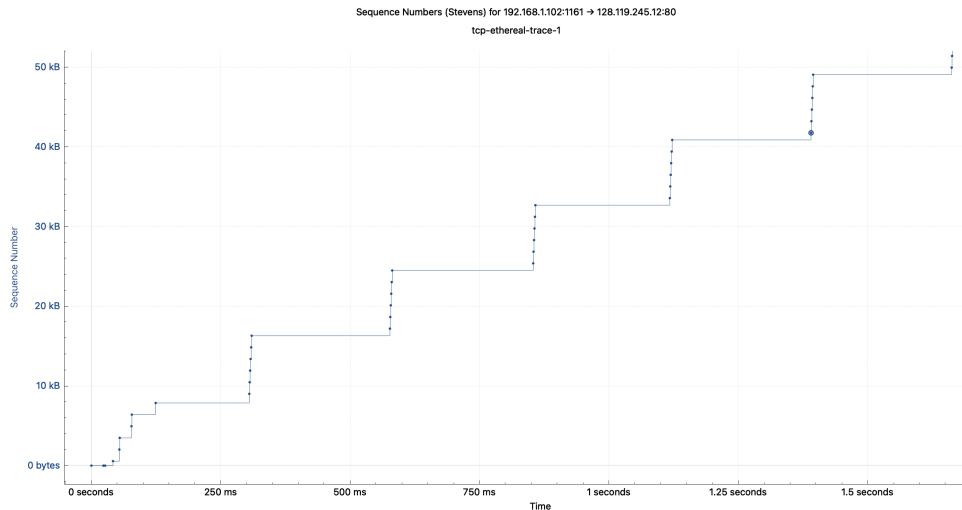


Figure 1: Sequence Number vs. Time (Stevens Graph) for the connection to gaia.cs.umass.edu

Analysis of the Graph: Based on the Time-Sequence (Stevens) graph shown in Figure 1, we can identify the phases of TCP congestion control:

- **Slow Start Phase:** The Slow Start phase is visible at the very beginning of the connection ($t \approx 0$ to $t \approx 0.6$ seconds). In this region, the vertical “steps” (representing bursts of data segments) increase in height rapidly. This indicates exponential growth in the congestion window, as the sender doubles the number of segments sent after each successful Round Trip Time (RTT).
- **Congestion Avoidance Phase:** Around $t \approx 0.55$ seconds (sequence number ≈ 24505), the behavior changes. The vertical steps become uniform in height and the slope becomes linear rather than exponential. This marks the transition to Congestion Avoidance, where the congestion window grows additively (typically increasing by 1 MSS per RTT) rather than multiplicatively.
- **Measured vs. Idealized Behavior:** While idealized textbook graphs show smooth curves, the measured data exhibits a distinct “staircase” pattern.
 - The **vertical lines** represent the transmission delay: the sender pushing a full window of segments onto the wire back-to-back.
 - The **horizontal flat lines** represent the propagation delay and queuing delay: the sender stopping and waiting for ACKs to return before the window slides forward to allow new transmissions.

14. Same as in the response to part 13.

1.2 UDP Lab

1. **Number of Fields in the UDP Header:** There are exactly **4 fields** in the UDP header. These fields are:
 - Source Port
 - Destination Port
 - Length
 - Checksum

2. **Length of UDP Header Fields:** When selecting any of the four fields in the details pane, Wireshark highlights exactly four hexadecimal digits in the raw packet bytes.

Since two hexadecimal digits correspond to one byte (8 bits), four hex digits represent **2 bytes**. Therefore, the length of each field is as follows:

- **Source Port:** 2 bytes
- **Destination Port:** 2 bytes
- **Length:** 2 bytes
- **Checksum:** 2 bytes

Consequently, the entire UDP header has a fixed, minimal length of exactly **8 bytes**.

3. Meaning and Verification of the UDP Length Field

- **Meaning of the Length Field:** The value in the Length field specifies the total number of bytes in the entire UDP segment. This total includes both the fixed 8-byte UDP header and the encapsulated application data (the payload).
- **Verification with Captured Packet:** This definition is clearly verified by the captured UDP packet in the trace:
 - The **Length** field in the header displays a value of **58**.
 - The packet details explicitly show that the **UDP payload** (the application data) is **50 bytes**.
 - We know the UDP header has a fixed size of **8 bytes**.

Adding the size of the header and the payload together (8 bytes + 50 bytes = 58 bytes) results in exactly the value shown in the Length field, confirming the claim.

4. Maximum UDP Payload Size

The maximum number of bytes that can be included in a UDP payload is **65,527 bytes**.

Derivation:

- The size of the UDP segment is dictated by the **Length** field in the UDP header.
- Because the Length field is exactly 2 bytes (16 bits) long, the maximum value it can hold is $2^{16} - 1 = 65,535$ bytes. This represents the maximum total length of the UDP segment (header + payload).
- To find the maximum payload size, we must subtract the fixed 8-byte UDP header from this total maximum length ($65,535 - 8 = 65,527$).

5. Largest Possible Source Port Number

The largest possible source port number is **65,535**.

Derivation: This limit is directly determined by the size of the **Source Port** field within the UDP header. As established from the packet inspection, this field is exactly 2 bytes (16 bits) long. Because port numbers are unsigned integers, the maximum value that can be represented by 16 bits is $2^{16} - 1$, which equals 65,535.

6. Protocol Number for UDP

By inspecting the encapsulating Internet Protocol (IPv4) header within the Wireshark trace, the Protocol field indicates that the payload is a UDP segment.

The protocol number assigned to UDP is:

- **Decimal Notation:** 17

- **Hexadecimal Notation:** 0x11

7. Relationship Between Port Numbers in a UDP Request/Reply Pair

- In the **first packet** (the request), the *Source Port* is an ephemeral port chosen by the client (e.g., 4334), and the *Destination Port* is the well-known port of the service running on the server (e.g., 161 for SNMP).
- In the **second packet** (the reply), the *Source Port* becomes the server's well-known port (161), and the *Destination Port* becomes the client's original ephemeral port (4334).

Conclusion: The relationship is a direct swap: $\text{Source Port}_{\text{request}} = \text{Destination Port}_{\text{reply}}$, and $\text{Destination Port}_{\text{request}} = \text{Source Port}_{\text{reply}}$. This swapping ensures that the server's response is routed back to the exact application process on the client that initiated the request.

2 Congestion Control Algorithm Comparison

2.1 Window Size vs. Time Plots

To analyze the behavior of the congestion control algorithms, Wireshark's TCP Stream Window Scaling graphs were utilized.

It is important to note a specific artifact of the network simulation environment. In real-world TCP captures, the green line in this graph represents the Advertised Receiver Window (*rwnd*), which typically remains relatively flat, while the blue line represents Unacked Bytes. However, to facilitate visualization, the simulator artificially injects the sender's internal Congestion Window (*cwnd*) directly into the TCP header's Window Size field. Consequently, the green line in the generated plots acts as a direct, 1-to-1 map of the *cwnd*'s mechanical behavior over time.

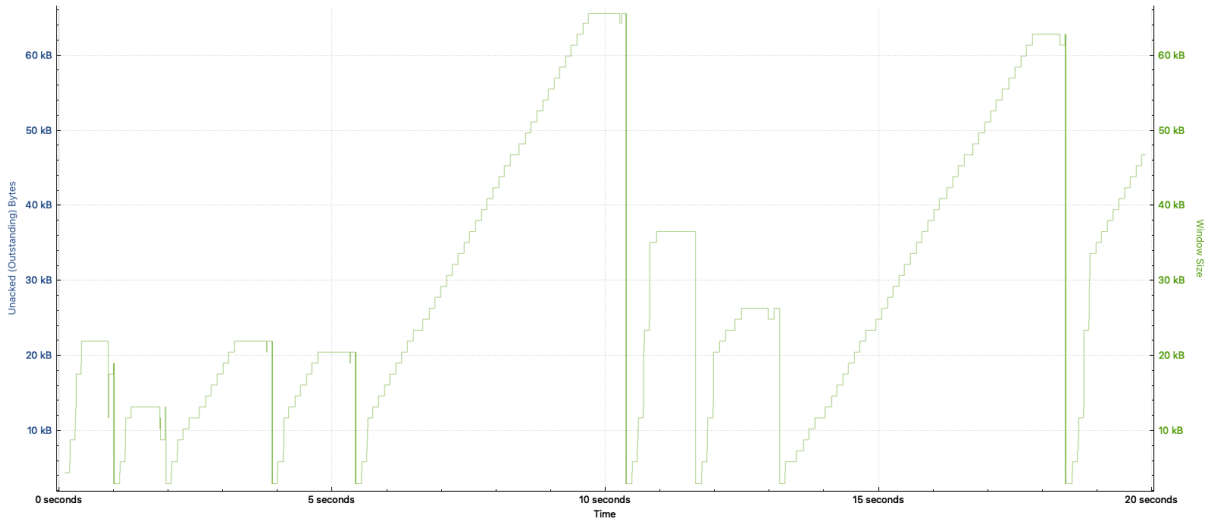


Figure 2: TCP Old Tahoe: Window Size vs. Time. The green line perfectly traces the algorithm's Slow Start phases, linear Congestion Avoidance, and severe drops to near-zero upon packet loss.

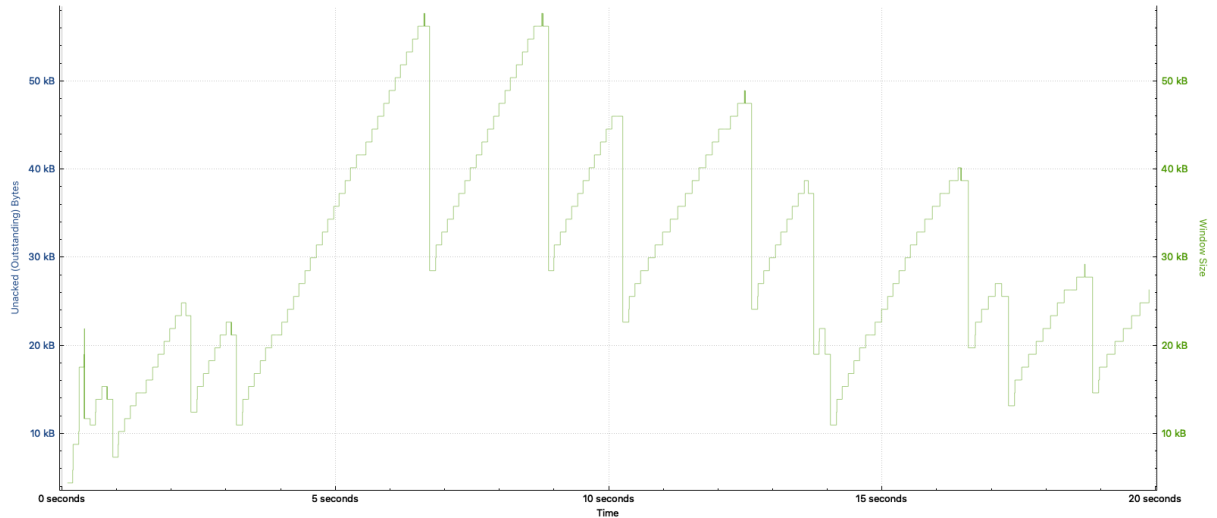


Figure 3: TCP Reno: Window Size vs. Time. Displays the classic additive-increase/multiplicative-decrease (AIMD) “sawtooth” pattern, utilizing Fast Recovery to halve the window rather than dropping to zero.

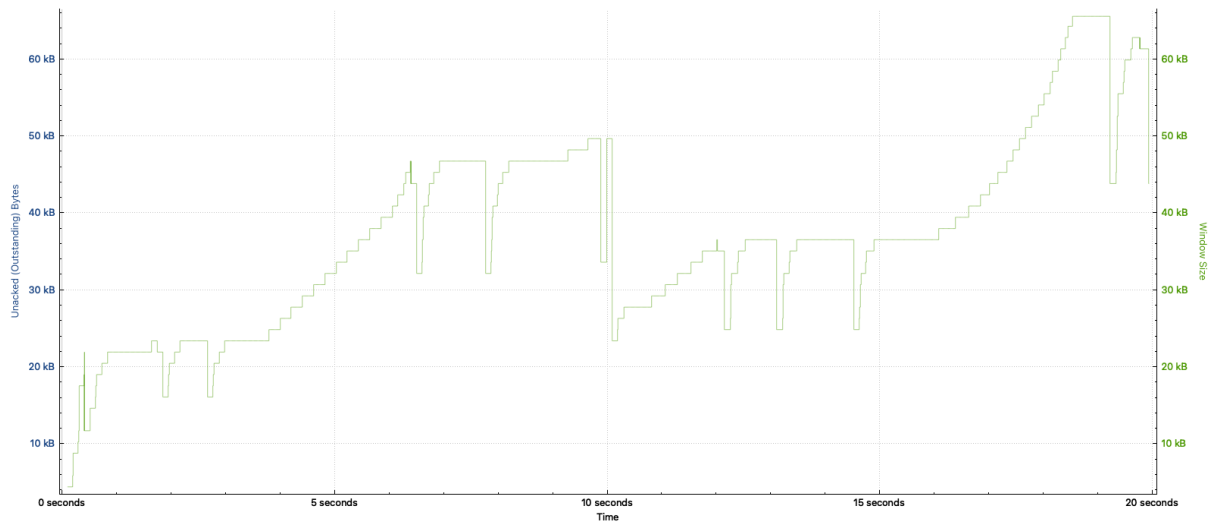


Figure 4: TCP Cubic: Window Size vs. Time. Displays rapid cubic recovery toward the previous maximum window size, followed by a flattened probing phase to maintain high capacity.

2.2 Algorithmic Comparison

The graphical outputs reveal the evolutionary improvements in TCP’s handling of packet loss:

- **TCP Old Tahoe:** As seen in Figure 2, Tahoe implements the most aggressive penalty for network congestion. Upon detecting loss, it blindly reduces its *cwnd* all the way to 1 Maximum Segment Size (MSS) and re-enters Slow Start, resulting in deep, vertical plunges in the window size.
- **TCP Reno:** Reno introduces Fast Recovery. Instead of dropping to 1 MSS upon receiving triple duplicate ACKs, it halves the window size ($cwnd = cwnd/2$) and performs a linear additive increase, creating a classic “sawtooth” pattern that recovers faster than Tahoe (Figure 3).

- **TCP Cubic:** Instead of a linear increase, Cubic’s window growth is dictated by a cubic function. It rapidly scales the window back toward the previous peak, flattens out to probe carefully near the maximum threshold, and avoids the drastic, constant halving seen in Reno (Figure 4).

2.3 Link Capacity Utilization

Based on the underlying mechanics and observed data, **TCP Cubic** is the algorithm that most effectively utilizes the link capacity to its maximum.

To illustrate why standard AIMD algorithms struggle with link utilization, we can observe the throughput graphs below. The severe window drops in Tahoe and Reno directly translate to massive plunges in data throughput (valleys), causing the link to sit highly underutilized during recovery phases.

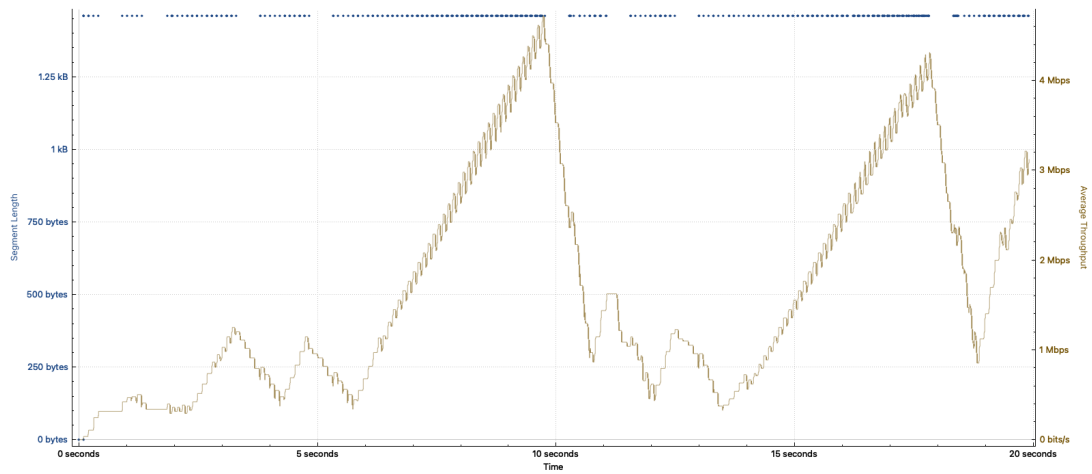


Figure 5: Tahoe Throughput

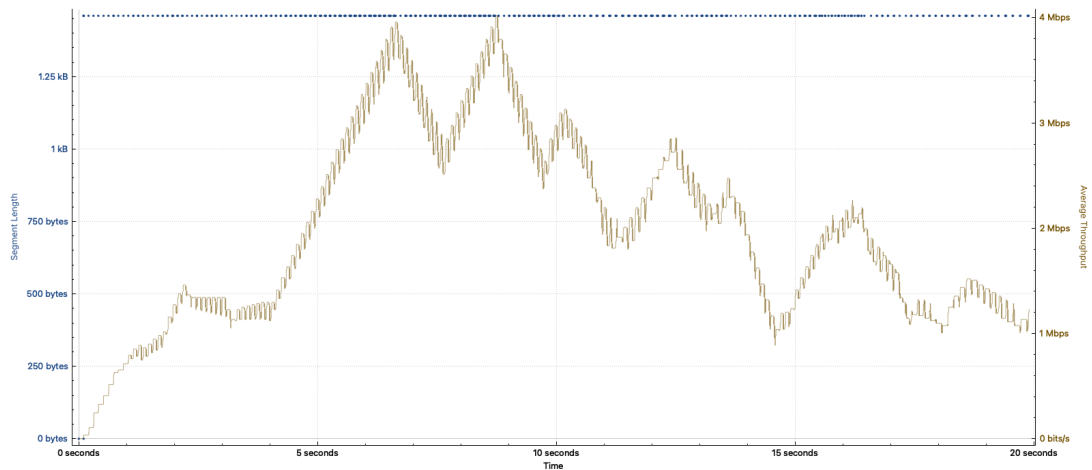


Figure 6: Reno Throughput

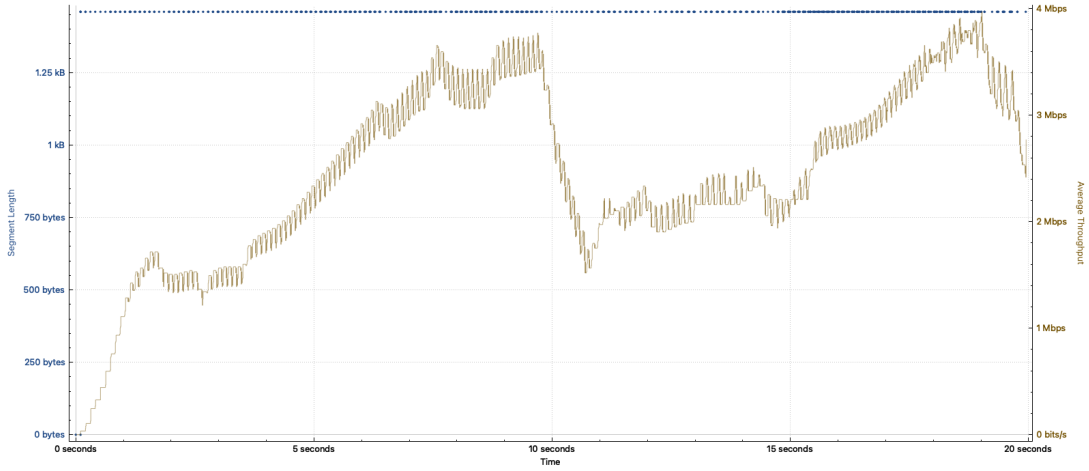


Figure 7: Cubic Throughput

Because Cubic uses its mathematical function to rapidly return to the known network capacity and spends the majority of its time operating very close to that threshold (the flattened part of the cubic curve), it avoids these deep throughput “valleys”.

2.4 Summary of Performance Metrics

The simulated performance statistics confirm Cubic’s superior link utilization. By maintaining a consistently high window size, Cubic pushes more total data across the link, resulting in the highest average throughput among the three variants.

Congestion Algorithm	Total Bytes Transferred	Average Throughput
TCP Old Tahoe	11,306,812 Bytes	4,522 kbps
TCP Reno	16,326,392 Bytes	6,530 kbps
TCP Cubic	17,348,352 Bytes	6,939 kbps

Table 2: Comparison of total data transfer and throughput across the simulated connection.

3 Speed Test using iPerf3

3.1 Methodology and Environment

To rigorously measure the maximum achievable bandwidth across different geographic locations, an international Wide Area Network (WAN) test was established. Instead of a standard local area network, this measurement was conducted between a local host in Kanpur, India, and a remote cloud-hosted Virtual Machine in East Asia. This setup provides a realistic representation of long-distance IP routing and real-world internet bandwidth constraints.

- **Client Node:** Local macOS environment (Kanpur).
- **Server Node:** Remote Windows Cloud Virtual Machine (East Asia, IP: 104.214.181.100).
- **Connection:** iPerf3 server listening on default TCP port 5201. The cloud Network Security Group (NSG) was configured to allow inbound traffic to facilitate the connection across the WAN.

3.2 Test 1: Standard TCP Throughput (Uplink)

The primary test measured the standard TCP throughput, representing the uplink capability from the local client to the remote server. The test was executed using the standard `iperf3 -c 104.214.181.100` command and ran for the default duration of 10 seconds.

```
Connecting to host 104.214.181.100, port 5201
[ 5] local 172.23.5.133 port 52530 connected to 104.214.181.100 port 5201
[ ID] Interval      Transfer    Bitrate
[ 5] 0.00-1.01    sec  2.25 MBytes 18.8 Mbits/sec
[ 5] 1.01-2.01    sec 10.2 MBytes 86.0 Mbits/sec
[ 5] 2.01-3.01    sec  640 KBytes  5.24 Mbits/sec
[ 5] 3.01-4.01    sec  5.62 MBytes 47.2 Mbits/sec
[ 5] 4.01-5.00    sec  6.50 MBytes 54.8 Mbits/sec
[ 5] 5.00-6.01    sec  7.25 MBytes 60.5 Mbits/sec
[ 5] 6.01-7.01    sec  7.75 MBytes 65.0 Mbits/sec
[ 5] 7.01-8.00    sec  6.25 MBytes 52.4 Mbits/sec
[ 5] 8.00-9.01    sec  6.25 MBytes 52.4 Mbits/sec
[ 5] 9.01-10.01   sec  6.50 MBytes 54.5 Mbits/sec
-----
[ ID] Interval      Transfer    Bitrate
[ 5] 0.00-10.01   sec  59.2 MBytes 49.7 Mbits/sec
[ 5] 0.00-10.13   sec  58.4 MBytes 48.4 Mbits/sec
sender
receiver
```

Figure 8: Test 1: Client-side output (macOS)

```
Server listening on 5201 (test #1)
-----
Accepted connection from 14.139.38.152, port 52529
[ 5] local 172.17.0.4 port 5201 connected to 14.139.38.152 port 52530
[ ID] Interval      Transfer    Bitrate
[ 5] 0.00-1.02    sec  1.00 MBytes  8.26 Mbits/sec
[ 5] 1.02-2.00    sec  8.75 MBytes 74.5 Mbits/sec
[ 5] 2.00-3.01    sec  1.12 MBytes  9.33 Mbits/sec
[ 5] 3.01-4.01    sec  6.38 MBytes 53.3 Mbits/sec
[ 5] 4.01-5.01    sec  6.25 MBytes 52.4 Mbits/sec
[ 5] 5.01-6.00    sec  6.88 MBytes 58.5 Mbits/sec
[ 5] 6.00-7.00    sec  7.88 MBytes 65.9 Mbits/sec
[ 5] 7.00-8.00    sec  6.00 MBytes 50.3 Mbits/sec
[ 5] 8.00-9.01    sec  6.75 MBytes 56.2 Mbits/sec
[ 5] 9.01-10.01   sec  6.62 MBytes 55.5 Mbits/sec
[ 5] 10.01-10.13  sec   768 KBytes 54.0 Mbits/sec
-----
[ ID] Interval      Transfer    Bitrate
[ 5] 0.00-10.13   sec  58.4 MBytes 48.4 Mbits/sec
receiver
```

Figure 9: Test 1: Server-side output (Windows VM)

3.3 Test 2: Reverse TCP Throughput (Downlink)

To provide a comprehensive analysis, a second test was executed using the reverse flag (`iperf3 -c 104.214.181.100 -R`). In this mode, the client connects to the server, but the server actively sends the data back to the client. This accurately measures the downlink bandwidth from the East Asian VM to the local machine in Kanpur.

```

Connecting to host 104.214.181.100, port 5201
Reverse mode, remote host 104.214.181.100 is sending
[ 5] local 172.23.5.133 port 52631 connected to 104.214.181.100 port 5201
[ ID] Interval          Transfer      Bitrate
[ 5] 0.00-1.01      sec    384 KBytes  3.13 Mbits/sec
[ 5] 1.01-2.00      sec    640 KBytes  5.25 Mbits/sec
[ 5] 2.00-3.00      sec     0.00 Bytes  0.00 bits/sec
[ 5] 3.00-4.01      sec    128 KBytes  1.05 Mbits/sec
[ 5] 4.01-5.01      sec    128 KBytes  1.05 Mbits/sec
[ 5] 5.01-6.01      sec    256 KBytes  2.10 Mbits/sec
[ 5] 6.01-7.00      sec    256 KBytes  2.10 Mbits/sec
[ 5] 7.00-8.01      sec     0.00 Bytes  0.00 bits/sec
[ 5] 8.01-9.01      sec    128 KBytes  1.05 Mbits/sec
[ 5] 9.01-10.01     sec    256 KBytes  2.10 Mbits/sec
-----
[ ID] Interval          Transfer      Bitrate
[ 5] 0.00-10.12     sec   2.38 MBytes  1.97 Mbits/sec
[ 5] 0.00-10.01     sec   2.12 MBytes  1.78 Mbits/sec
sender
receiver

```

Figure 10: Test 2: Client-side output (macOS)

```

-----
Server listening on 5201 (test #2)
-----
Accepted connection from 14.139.38.152, port 52630
[ 5] local 172.17.0.4 port 5201 connected to 14.139.38.152 port 52631
[ ID] Interval          Transfer      Bitrate
[ 5] 0.00-1.00      sec    512 KBytes  4.18 Mbits/sec
[ 5] 1.00-2.01      sec    768 KBytes  6.25 Mbits/sec
[ 5] 2.01-3.00      sec     0.00 Bytes  0.00 bits/sec
[ 5] 3.00-4.01      sec    128 KBytes  1.04 Mbits/sec
[ 5] 4.01-5.01      sec    128 KBytes  1.04 Mbits/sec
[ 5] 5.01-6.01      sec    128 KBytes  1.05 Mbits/sec
[ 5] 6.01-7.00      sec    384 KBytes  3.18 Mbits/sec
[ 5] 7.00-8.01      sec     0.00 Bytes  0.00 bits/sec
[ 5] 8.01-9.01      sec    128 KBytes  1.05 Mbits/sec
[ 5] 9.01-10.01     sec    256 KBytes  2.12 Mbits/sec
[ 5] 10.01-10.12    sec     0.00 Bytes  0.00 bits/sec
-----
[ ID] Interval          Transfer      Bitrate
[ 5] 0.00-10.12     sec   2.38 MBytes  1.97 Mbits/sec
sender

```

Figure 11: Test 2: Server-side output (Windows VM)

3.4 Results and Conclusion

The iPerf3 utility successfully demonstrated the bandwidth limitations and performance of a long-haul international link, revealing highly asymmetric routing behavior.

- **Uplink Performance (Test 1):** The connection transferred a total of **58.4 MBytes** over 10 seconds, achieving an average upload bandwidth of **48.4 Mbits/sec**.
- **Downlink Performance (Test 2):** The reverse test transferred a total of only **2.12 MBytes**, achieving a severely constrained average download bandwidth of **1.78 Mbits/sec**.

Analysis: The downlink was roughly 25x slower than the uplink, frequently dropping to 0.00 bits/sec. This extreme asymmetry highlights how geographic distance and distinct ISP routing heavily impact WAN TCP performance. The bottleneck likely stems from:

- Severe network congestion or packet loss on the East Asia-India return path.
- Aggressive outbound bandwidth throttling by the cloud provider’s data center.